COLLEGE OF ENGINEERING TRIVANDRUM

Department of Computer Science & Engineering

APJ Abdul Kalam Technological University

COURSE PROJECT REPORT

PBCST304 – OBJECT ORIENTED PROGRAMMING

SmartLib: A Multi-Tier Desktop College Library Management System

---

Submitted by:

Adhithyan K S
KTU ID: TVE23CS001
Semester 5, B.Tech CSE

Jenil Prince
KTU ID: TVE23CS064
Semester 5, B.Tech CSE

Course Coordinator:

Faculty in Charge
Department of CSE
College of Engineering Trivandrum

Submission Deadlines:

Soft Copy (GCR): 27 October 2026
Hard Copy (Bound): 28 October 2026

October 2026

Academic Year 2026–2027

Abstract

Academic libraries serve as the intellectual cornerstone of collegiate institutions, coordinating the acquisition, cataloging, circulation, and preservation of scholarly assets. Despite widespread digitization across universities, many institutional libraries continue to grapple with antiquated management software or error-prone manual ledgers. Common systemic drawbacks include cumbersome book circulation workflows, unmonitored overdue loans leading to substantial fee leakages, double-booking race conditions during manual reservations, and the complete absence of a secure, transparent self-service portal for enrolled students.

To resolve these challenges, this report presents **SmartLib**, a robust, desktop-based College Library Management System developed using Java SE 17, Swing/AWT graphical user interfaces, JDBC, and a relational MySQL database. Engineered in accordance with the curricular requirements of course **PBCST304 (Object Oriented Programming)**, SmartLib strictly adheres to multi-tier layered software architectural patterns, embodying rigorous Separation of Concerns (SoC) and the Dependency Inversion Principle (DIP). The architecture cleanly delineates responsibilities across four distinct layers: a responsive Presentation Layer employing Swing's `CardLayout` navigation model, a Business Service Layer encapsulating institutional policies, an abstracted Data Access Object (DAO) Layer centered around a generic `CrudDAO<T, ID>` base interface, and a persistent MySQL relational database.

SmartLib introduces innovative operational features that elevate collegiate library administration. A unified authentication mechanism with 12-round BCrypt cryptographic password hashing provides role-based access control for both Librarians and Students. Librarians benefit from an advanced Circulation Desk equipped with real-time Active Borrowings Management, which provides a live 10-column inventory of active loans, dynamic clock-aware overdue calculations (Fine = Days Overdue $\times$ Rs. 10), multi-criteria instant filtering, and single-click return processing. An automated Title Reservation subsystem eliminates concurrency conflicts through a First-In, First-Out (FIFO) queue that proactively triggers fulfillment notifications upon book returns. Concurrently, students are empowered through a dedicated self-service portal featuring catalog exploration, borrowing requests, and real-time fine visibility.

The software system was verified through an automated test suite comprising 43

unit test cases implemented in JUnit 5, employing mock DAOs and deterministic clock abstractions to guarantee rock-solid business logic integrity. The resulting application delivers an intuitive, secure, and production-grade library automation solution that demonstrates the practical application of object-oriented paradigms to enterprise software development.

Keywords: Object-Oriented Programming, Multi-Tier Architecture, Java Swing, JDBC, Data Access Object Pattern, BCrypt Authentication, Active Borrowings Tracking, JUnit 5 Testing.

Contents

Abstract ii

1 Introduction 1
- 1.1 Domain Background and Context . . . 1
- 1.2 Limitations of Legacy and Manual Systems . . . 1
- 1.3 Motivation for SmartLib . . . 2
- 1.4 Scope and Significance . . . 2

2 Problem Statement and Objectives 4
- 2.1 Problem Statement . . . 4
- 2.2 Project Objectives . . . 5
  - 2.2.1 Architectural Objectives . . . 5
  - 2.2.2 Functional Objectives . . . 5
  - 2.2.3 Quality and Verification Objectives . . . 6

3 Requirements Specification 7
- 3.1 Functional Requirements . . . 7
- 3.2 Non-Functional Requirements . . . 9
- 3.3 Hardware and Software Environment . . . 10
  - 3.3.1 Hardware Requirements . . . 10
  - 3.3.2 Software Environment and Dependency Matrix . . . 10

4 System Design 11
- 4.1 System Architecture . . . 11
  - 4.1.1 Layer Responsibilities . . . 11
- 4.2 Class Diagram . . . 12
- 4.3 Class Descriptions, Attributes, and Methods . . . 12
  - 4.3.1 Domain Models (`com.college.library.model` and `dto`) . . . 12
  - 4.3.2 Business Services (`com.college.library.service`) . . . 14
  - 4.3.3 Data Access Layer (`com.college.library.dao` and `dao.impl`) 15
- 4.4 Relationships Between Classes . . . 16
- 4.5 Database Design . . . 17

4.5.1 Entity-Relationship (ER) Diagram . . . 17
4.5.2 Relational Data Dictionary . . . 17
4.5.3 Database Normalization Analysis . . . 18
4.5.4 Accession ID vs. ISBN Architectural Distinction . . . 19
4.6 User Interface Design . . . 19
4.6.1 Design System and UITheme Framework . . . 19
4.6.2 Custom Table Interaction Engine (`TableSelectionHelper`) . . . 19

5 Implementation **20**
5.1 Development Platform and Toolchain . . . 20
5.2 Implementation of Object-Oriented Programming (OOP) Principles . . . 20
5.2.1 Encapsulation and Information Hiding . . . 20
5.2.2 Abstraction and Interface Segregation . . . 21
5.2.3 Inheritance Hierarchies . . . 21
5.2.4 Polymorphism and Dynamic Dispatch . . . 22
5.3 Database Connectivity and Defensive JDBC Engineering . . . 22
5.3.1 The `DatabaseConnection` Singleton Factory . . . 22
5.3.2 SQL Injection Mitigation and Resource Safety . . . 23
5.4 Critical Business Logic and Algorithms . . . 23
5.4.1 Clock-Aware Overdue Fine Calculation Engine . . . 23
5.4.2 Direct Book Issue Algorithm with Concurrency Safeguard . . . 24
5.4.3 Book Return and Reservation Fulfillment Trigger . . . 24
5.4.4 Automated KTU Registration ID Parser . . . 25

6 Testing and Test Cases **26**
6.1 Testing Methodology and Philosophy . . . 26
6.1.1 Isolation via Mocking and Clock Simulation . . . 26
6.2 Automated Test Suite Overview . . . 26
6.3 Detailed Test Cases and Results . . . 27
6.3.1 Authentication and Cryptography Test Cases . . . 27
6.3.2 Borrow and Circulation Desk Test Cases (`BorrowServiceTest`) 28
6.3.3 Student Service and Clearance Test Cases (`StudentServiceTest`) 29

7 Results **31**
7.1 Application Functionality Overview . . . 31
7.2 User Interface Walkthrough and Screenshots . . . 31
7.2.1 User Authentication and Registration Portal . . . 32
7.2.2 Executive Analytics Dashboard . . . 32
7.2.3 Book Inventory Management Panel . . . 32
7.2.4 Student Management and Clearance Panel . . . 32

7.2.5 Circulation Desk and Active Borrowings Panel 34
7.2.6 Reservation Queue Management Panel 34
7.2.7 Student Self-Service Dashboard 34
7.3 Sample Operational Scenarios and Traces 35
7.3.1 Scenario A: Direct Book Issue at Circulation Desk 36
7.3.2 Scenario B: Overdue Book Return with Fine Calculation and Reservation Trigger 37
7.3.3 Scenario C: Student Clearance Verification 37

8 Conclusion and Future Enhancements **39**
8.1 Summary of Achievements 39
8.2 Educational Outcomes and Retrospective 40
8.3 Future Enhancements 40

References **42**

Appendices **43**
A User Manual and Installation Instructions 43
A.1 Prerequisites Checklist 43
A.2 Database Initialization 43
A.3 Database Configuration 44
A.4 Building and Executing the Application 44
A.5 Default User Credentials 44
A.6 Troubleshooting Matrix 44
B Selected Source Code Listings 45
B.1 Generic Base Data Access Object (`CrudDAO.java`) 45
B.2 Circulation Desk and Dynamic Fine Engine (`BorrowService.java`) 45
B.3 Centralized Design System (`UITheme.java`) 49

List of Figures

4.1 SmartLib Multi-Tier Layered Architecture 12
4.2 UML Class Diagram of Core Domain Entities 13
4.3 Entity-Relationship (ER) Diagram of SmartLib Database 18

7.1 SmartLib User Authentication Portal (`LoginPanel`) 32
7.2 Librarian Executive Analytics Dashboard (`DashboardPanel`) 33
7.3 Book Catalog and Inventory Management Panel (`BookPanel`) 33
7.4 Student Management and Clearance Panel (`StudentPanel`) 34
7.5 Circulation Desk and Active Borrowings Management Panel (`BorrowPanel`) 35
7.6 Title Reservation Queue Management Panel (`ReservationPanel`) . . . 35
7.7 Student Self-Service Dashboard (`StudentDashboardPanel`) 36

List of Tables

3.1 Hardware Environment Specifications 10
3.2 Software Environment and Technical Stack 10

4.1 Domain Model Class Specifications 13
4.2 Business Service Layer Class Specifications 14
4.3 Data Access Layer Specifications 16

6.1 JUnit 5 Test Suite Execution Summary 27
6.2 Authentication and Cryptography Test Cases 27
6.3 Circulation and Borrowing Test Cases 28
6.4 Student Management and Clearance Test Cases 29

8.1 Default Seed Credentials for Verification 45
8.2 Common Setup Errors and Solutions 45

Chapter 1

Introduction

1.1 Domain Background and Context

In modern academic institutions, the library is not merely a repository of printed books, but an active academic hub supporting undergraduate and postgraduate research, technical curricula, and faculty scholarship. The operational backbone of any college library is its circulation desk, where catalog records are maintained, physical copies are loaned and returned, late fees are assessed, and student clearances are approved.

For decades, academic libraries operated on traditional ledger-based systems or primitive single-tier computer spreadsheets. While functional for smaller collections, such systems do not scale to contemporary engineering colleges that house tens of thousands of volumes across diverse technical domains and cater to thousands of enrolled students. Effective management requires rigorous inventory control, automated transaction logging, strict enforcement of institutional borrowing rules, and auditable monetary fine calculations.

1.2 Limitations of Legacy and Manual Systems

Field analysis of conventional library workflows reveals several severe structural deficiencies:

1. **Manual Book Ledgers and Human Inaccuracy:** Manual physical registers are prone to transcription errors, illegible handwriting, and misplaced loan records, resulting in untracked book losses.

2. **Opaque Loan Overdue Visibility:** Circulation desks often lack immediate visibility into which physical copies are overdue. Without live computational indicators, librarians must manually cross-reference return dates against daily calendars, leading to significant delays and revenue leakage in overdue fines.

3. **Concurrency and Reservation Conflicts:** When popular textbooks (e.g., core engineering texts) are in high demand, manual reservation tracking fails. Students frequently experience double-booking or arbitrary queue jumping due to the lack of an automated, equitable First-In, First-Out (FIFO) reservation pipeline.

4. **Absence of Student Self-Service:** Conventional systems relegate students to passive consumers who must physically visit the circulation desk to inquire about book availability, loan deadlines, or accumulated dues. This creates unnecessary administrative bottlenecks during exam cycles and semester clearances.

5. **Monolithic and Fragile Codebases:** Existing computer applications in this domain are frequently structured as monolithic scripts with tight coupling between user interfaces and raw SQL queries, violating fundamental software engineering principles and preventing modular maintenance or future expansion.

1.3 Motivation for SmartLib

The primary motivation behind developing **SmartLib** is to bridge the gap between academic theory in Object-Oriented Programming (OOP) and real-world enterprise desktop software engineering. Under the curriculum of **PBCST304 (Object Oriented Programming)**, students are tasked with applying core object-oriented paradigms—such as abstraction, encapsulation, inheritance, polymorphism, design patterns, and defensive programming—to design a comprehensive software solution.

SmartLib is conceived as an industrial-grade, multi-tier desktop application built on modern Java SE 17, Swing/AWT, JDBC, and MySQL. It introduces a decoupled four-tier architecture that cleanly isolates presentation logic, domain models, business services, and database persistence. By introducing specialized features such as live Active Borrowings oversight, dynamic fine calculation engines, automated reservation fulfillment, and cryptographic role-based authentication, SmartLib eliminates manual bottlenecks and provides a seamless digital experience for both librarians and students.

1.4 Scope and Significance

The operational scope of SmartLib encompasses the full life-cycle of library circulation:

- **Administrative Authority (Librarian Operations):** Full cataloging of book titles categorized by academic disciplines; student enrollment with automatic batch/branch derivation from KTU registration IDs; direct book issue and return desk; real-time active loan monitoring with overdue highlighting; pending

reservation fulfillment; online borrow request approval/rejection; and student dues clearance verification.

- **Student Self-Service Portal:** Secure student registration and credential login; searchable catalog lookup with real-time availability status; one-click title reservation for currently issued books; online borrow requests; and an active dashboard providing full visibility into issued books, due dates, and fine liabilities.

- **Educational and Architectural Significance:** Beyond its operational utility, SmartLib serves as an exemplar implementation of the Data Access Object (DAO) pattern, custom Swing UI design systems, ACID transaction management over JDBC, and automated unit testing methodologies using JUnit 5.

Chapter 2

Problem Statement and Objectives

2.1 Problem Statement

In academic institutions affiliated with APJ Abdul Kalam Technological University (KTU), managing the circulation of library literature among hundreds of engineering students requires rigorous tracking, strict adherence to loan durations, and impartial enforcement of overdue fine regulations. Conventional manual registers and ad-hoc single-tier software applications suffer from critical operational failures:

1. **Circulation Desk Opacity:** Librarians lack real-time computational oversight of currently loaned books, which makes calculating overdue days and accrued fines tedious, error-prone, and inconsistent.

2. **Data Inconsistency and Concurrency Anomalies:** Without strict relational constraints and database-level transaction locking, concurrent operations (such as attempting to issue an already loaned book or reserving a title without checking active inventory) create corrupt data states.

3. **Inadequate Security and Access Control:** Many departmental library systems store passwords in plain text or lack clear separation between administrative privileges and student viewing rights.

4. **Architectural Fragility:** Codebases that mix database SQL queries directly inside GUI event listeners violate the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP), making code maintenance, feature enhancement, and unit testing virtually impossible.

Therefore, the core engineering challenge is to design, implement, and rigorously test a multi-tier, object-oriented desktop library management system that provides synchronized, role-based workflows for librarians and students, enforces database integrity, computes overdue fines dynamically, and adheres strictly to industry-standard design patterns.

2.2 Project Objectives

To address the defined problem statement, the SmartLib project was structured around the following concrete objectives:

2.2.1 Architectural Objectives

- **Tiered Separation of Concerns:** Implement a rigorous Multi-Tier Layered Architecture consisting of Presentation Layer (`gui`), Business Service Layer (`service`), Data Access Object Layer (`dao` and `dao.impl`), and Persistent Relational Database (MySQL).
- **Interface-Driven Design:** Abstract all persistence operations behind generic interfaces (`CrudDAO<T, ID>`), ensuring the high-level business services remain completely decoupled from specific database queries and JDBC drivers.
- **Defensive Database Engineering:** Guarantee ACID compliance by managing database connections through a dedicated connection manager, enforcing parameterized SQL queries via `PreparedStatement` to eliminate SQL injection vulnerabilities, and managing connection lifecycles via Java `try-with-resources`.

2.2.2 Functional Objectives

- **Dual Role-Based Authentication:** Provide isolated authentication pipelines for Librarians and Students, securing credentials using industry-standard 12-round BCrypt salt hashing.
- **Automated Circulation Desk & Active Borrowings Tracking:** Equip the circulation desk with a live 10-column tracking grid of all active loans, dynamic overdue day calculation, clock-aware overdue fine estimation (Rs. 10/day), multi-column search, and direct single-click returns.
- **Automated Reservation Pipeline:** Establish a FIFO reservation queue that permits students to reserve titles currently checked out, automatically alerting the librarian to fulfill pending reservations when the title is returned.
- **Student Self-Service Portal:** Provide students with direct catalog browsing, real-time availability inquiries, online borrow request submissions, and clear oversight of personal loan records and accumulated dues.
- **Institutional Data Integration:** Automate student validation and enrollment parsing based on standardized KTU registration ID formatting (e.g., regex pat-

tern `^[A-Z]{3}[0-9]{2}[A-Z]{2}[0-9]{3}$`), extracting year, batch, and engineering branch dynamically.

2.2.3 Quality and Verification Objectives

- **Automated Unit Testing:** Construct a comprehensive suite of automated unit tests using JUnit 5, validating domain services against mock DAO stubs and simulated temporal clocks without requiring an active external database connection.
- **Industrial Ergonomics:** Build a professional, polished Java Swing user interface utilizing a cohesive design theme (`UITheme`), clean typography, card layout navigation, and custom table selection managers.

Chapter 3

Requirements Specification

3.1 Functional Requirements

The functional requirements specify the actions, operations, and behaviors that the SmartLib software system must support.

FR-1: User Authentication and Authorization:

- The system must provide distinct login screens for Librarians and Students.
- Student registration must enforce validation of standard KTU Registration IDs using regex pattern `^[A-Z]{3}[0-9]{2}[A-Z]{2}[0-9]{3}$`.
- All passwords must be salted and hashed using 12 rounds of BCrypt before persistence; plain text passwords must never be stored.
- Sessions must maintain user context across screen transitions and prohibit unauthorized administrative access to student sessions.

FR-2: Book Catalog and Inventory Management:

- Librarians must be able to add, update, delete, and view books.
- Each physical book copy must be tracked via a unique `accession_id` (e.g., `B1001`), while sharing bibliographic attributes such as ISBN, title, author, and academic category.
- Book inventory status must transition strictly between defined enumerations: `AVAILABLE`, `ISSUED`, `LOST`, and `DAMAGED`.
- The catalog must support real-time text-based search across titles, authors, ISBNs, and categories.

FR-3: Student Membership and Clearance Management:

- Librarians must be able to enroll students, edit profiles, and view academic standings.
- Entering a valid KTU ID must automatically parse and populate the student's admission year, academic batch, and engineering branch.
- The system must provide a clearance check mechanism that inspects whether a student has any outstanding unreturned books or unpaid fines before granting graduation/transfer clearance.

FR-4: Circulation Desk and Active Borrowings Management:

- Librarians must be able to issue available books directly to enrolled students upon validating student ID and accession ID.
- The default loan period must be configured to 14 calendar days from the date of issue.
- The circulation desk must provide a live 10-column inventory table displaying all currently active borrowings (Transaction ID, Book ID, Title, Author, Student ID, Student Name, Issue Date, Due Date, Days Overdue, and Accrued Fine).
- The system must dynamically compute overdue duration ($\max(0, \text{Current Date} - \text{Due Date})$) and accrued fine (Days Overdue $\times$ Rs. 10.00) in real time.
- Librarians must be able to filter active loans by status (`All Active`, `Overdue`, `Due Today`) and execute direct one-click returns from selected table rows.

FR-5: Borrow Request Workflow:

- Students must be able to browse available books and submit digital borrow requests through their self-service dashboard.
- Librarians must be able to view all pending requests, approve valid requests (which transitions the book to `ISSUED` and generates an active transaction), or reject requests with appropriate reason logging.

FR-6: Title Reservation Queue Management:

- Students must be able to place reservations on books that are currently issued or unavailable, referenced by ISBN.
- Reservations must be sequenced in a strict First-In, First-Out (FIFO) queue.

- When an issued book is returned at the circulation desk, the system must detect any pending reservations for that ISBN and display an immediate notification prompting the librarian to hold the book for the reserving student.

FR-7: Executive Analytics and Dashboard:

- The Librarian dashboard must aggregate and display real-time Key Performance Indicators (KPIs): Total Books in Catalog, Active Borrowings Count, Enrolled Active Students, and Overdue Books Count.
- The dashboard must present a live summary table of the most popular books based on historical borrowing frequency.

3.2 Non-Functional Requirements

NFR-1: Performance and Latency:

- All database search and retrieval operations must execute within 500 milliseconds for catalog collections exceeding 10,000 records.
- UI panel transitions managed by `CardLayout` must be instantaneous (< 100 milliseconds).

NFR-2: Security and Data Integrity:

- Complete protection against SQL Injection attacks through universal enforcement of parameterized `PreparedStatement` queries across all DAO implementations.
- Passwords protected using BCrypt hashing with a computational cost factor of 12 rounds.
- Externalization of database credentials into `database.properties`, which is excluded from version control systems.

NFR-3: Usability and Ergonomics:

- Consistent visual styling and typography across all screens powered by a centralized design system (`UITheme`).
- Visual differentiation using color-coded status pills (e.g., Green for Available, Red for Overdue, Orange for Pending).
- Robust table selection and deselection handling via a dedicated `TableSelectionHelper`.

NFR-4: Reliability, Maintainability, and Testability:

- Adherence to SOLID principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.
- High automated test coverage with JUnit 5, isolating business logic via mock DAOs and temporal clock injection.

3.3 Hardware and Software Environment

3.3.1 Hardware Requirements

Table 3.1: Hardware Environment Specifications

Component	Minimum Requirement	Recommended Specification
Processor	Dual-Core x86_64 (1.8 GHz)	Quad-Core Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB or higher
Storage	500 MB free disk space	2 GB SSD storage
Display	1024×768 resolution	1920×1080 Full HD resolution
Network	Localhost TCP loopback	100 Mbps Local Area Network

3.3.2 Software Environment and Dependency Matrix

Table 3.2: Software Environment and Technical Stack

Layer / Component	Technology	Version / Description
Runtime Environment	Java SE Development Kit (JDK)	Version 17 LTS (Adoptium / Or OpenJDK)
GUI Framework	Java Swing & AWT	Native standard library (`javax.sw`
Database Engine	MySQL Community Server	Version 8.0 or higher
Build & Dependency Tool	Apache Maven	Version 3.8+ (Maven Wrapper `m` bundled)
Testing Framework	JUnit Jupiter (JUnit 5)	Version 5.10.0
JDBC Driver	MySQL Connector/J	`com.mysql:mysql-connector-j:9`
Security Library	jBCrypt	`org.mindrot:jbcrypt:0.4`
Operating System	Cross-Platform	Linux (Ubuntu 22.04/24.04), Wind 10/11

Chapter 4

System Design

4.1 System Architecture

SmartLib is designed strictly according to the **Multi-Tier Layered Architectural Pattern**, embodying Separation of Concerns (SoC) and the Dependency Inversion Principle (DIP). Each layer maintains specific responsibilities and communicates strictly with adjacent layers through well-defined contracts.

4.1.1 Layer Responsibilities

1. **Presentation Layer (`gui`):** Exclusively manages the visual components and user interactions using Java Swing and AWT. It contains zero SQL queries or business calculation logic. View transitions are driven by a centralized `CardLayout` router managed by `MainApplication`.

2. **Business Service Layer (`service`):** Encapsulates institutional policies, operational constraints, and domain algorithms. It coordinates transactions, verifies loan limits, computes clock-aware fines, manages FIFO reservation fulfillment, and communicates strictly through DAO interface references.

3. **Data Access Object Layer (`dao` and `dao.impl`):** Implements the DAO design pattern to insulate business services from database specifics. It encapsulates raw SQL queries, prepares statements, and maps relational result sets into Java domain entities.

4. **Persistence Layer (MySQL):** Manages data storage, relational integrity, primary and foreign key constraints, indexes, and ACID guarantees.

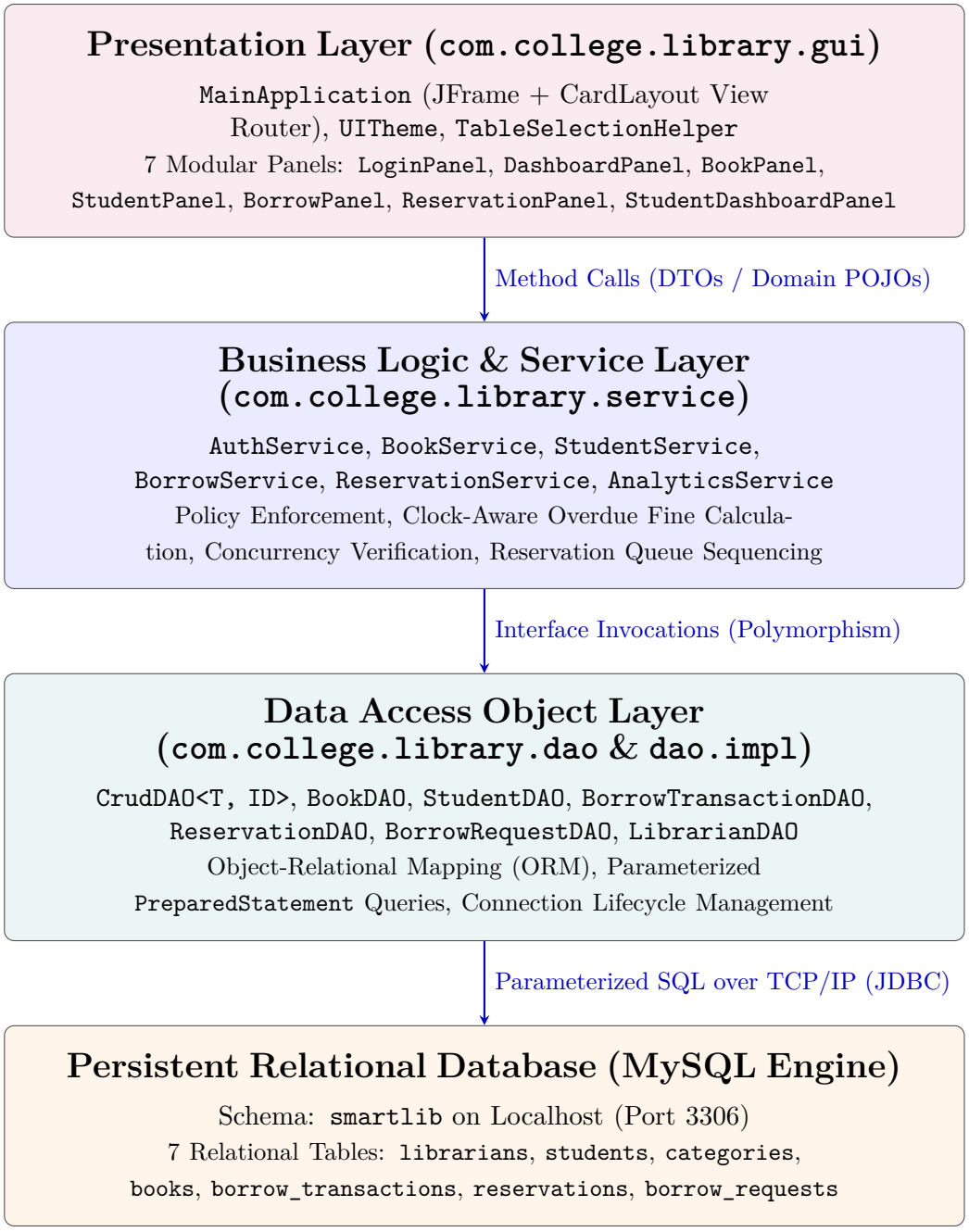


Figure 4.1: SmartLib Multi-Tier Layered Architecture

4.2 Class Diagram

The static structure of SmartLib is centered around domain models, generic data access interfaces, and business services.

4.3 Class Descriptions, Attributes, and Methods

4.3.1 Domain Models (com.college.library.model and dto)

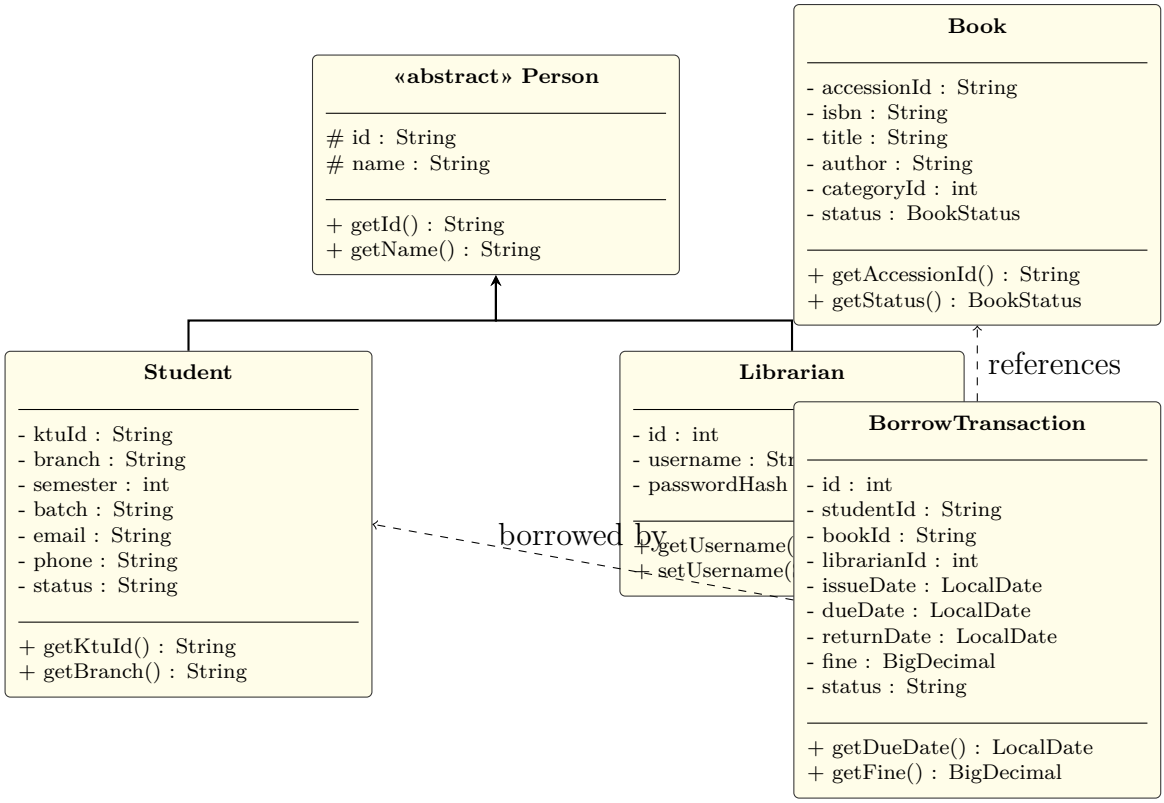


Figure 4.2: UML Class Diagram of Core Domain Entities

Table 4.1: Domain Model Class Specifications

Class Name	Attributes	Methods and Responsibilities
`Person` (abstract)	`id`, `name`	Base entity encapsulating common human identity attributes.
`Librarian`	`id`, `username`, `passwordHash`, `name`	Extends `Person`. Represents administrative users with circulation privileges.
`Student`	`ktuId`, `passwordHash`, `name`, `branch`, `semester`, `batch`, `email`, `phone`, `status`	Extends `Person`. Represents institutional student patrons with KTU identifiers.
`Book`	`accessionId`, `isbn`, `title`, `author`, `categoryId`, `status`	Represents individual physical catalog copies. Encapsulates status enumerations (`AVAILABLE`, `ISSUED`, etc.).

Table 4.1 continued from previous page

Class Name	Attributes	Methods and Responsibilities
`Category`	`id`, `name`	Academic subject categorization taxonomy (e.g., Computer Science, Mathematics).
`BorrowTransaction`	`id`, `studentId`, `bookId`, `librarianId`, `issueDate`, `dueDate`, `returnDate`, `fine`, `status`	Represents physical loan agreements, tracking critical due dates and financial liabilities.
`Reservation`	`id`, `studentId`, `bookIsbn`, `requestDate`, `status`	Models FIFO reservation queues for titles checked out by other patrons.
`BorrowRequest`	`id`, `studentId`, `bookId`, `requestDate`, `status`	Models online book issue requests submitted from student dashboards.
`ActiveBorrowingDTO`	10 fields: `txId`, `bookId`, `title`, `author`, `studentId`, `studentName`, `issueDate`, `dueDate`, `daysOverdue`, `currentFine`	Immutable Data Transfer Object designed to carry aggregated active loan details directly to the circulation table.

4.3.2 Business Services (`com.college.library.service`)

Table 4.2: Business Service Layer Class Specifications

Service Name	DAO Dependencies	Core Methods and Responsibilities
`AuthService`	`LibrarianDAO`, `StudentDAO`	`loginLibrarian()`, `loginStudent()`, `registerStudent()`. Manages BCrypt validation and role identification.

Table 4.2 continued from previous page

Service Name	DAO Dependencies	Core Methods and Responsibilities
BookService	BookDAO	addBook(), updateBook(), deleteBook(), searchBooks(). Enforces catalog rules and unique accession constraints.
StudentService	StudentDAO, BorrowTransactionDAO, ReservationDAO, BorrowRequestDAO	registerStudent(), updateStudent(), getStudentDues(), verifyClearance(). Computes academic clearances and outstanding debts.
BorrowService	BorrowTransactionDAO, BookDAO, StudentDAO, ReservationDAO, BorrowRequestDAO	issueBook(), returnBook(), getAllActiveBorrowings(), calculateOverdueFine(). Core circulation engine with clock-aware fine computation and reservation triggers.
ReservationService	ReservationDAO, StudentDAO, BookDAO	createReservation(), cancelReservation(), fulfillReservation(). Manages FIFO reservation lifecycle.
AnalyticsService	BookDAO, StudentDAO, BorrowTransactionDAO	getTotalBooks(), getActiveLoansCount(), getOverdueCount(), getPopularBooks(). Aggregates executive dashboard metrics.

4.3.3 Data Access Layer (com.college.library.dao and dao.impl)

Table 4.3: Data Access Layer Specifications

Interface / Impl	**Generic Base**	**Key Query Methods**
`CrudDAO<T, ID>`	N/A	`save(T)`, `findById(ID)`, `findAll()`, `update(T)`, `delete(ID)`. Generic CRUD contract.
`BookDAO` / `Impl`	`CrudDAO<Book, String>`	`findByIsbn()`, `search()`, `updateStatus()`, `getPopularBooks()`. Manages accession inventory.
`StudentDAO` / `Impl`	`CrudDAO<Student, String>`	`findByKtuId()`, `updatePassword()`, `countActiveStudents()`.
`BorrowTransactionDAO` / `Impl`	`CrudDAO<BorrowTransaction, Integer>`	`findActiveByStudentId()`, `findActiveByBookId()`, `findAllActiveTransactionsWithDetails()`, `closeTransaction()`.
`ReservationDAO` / `Impl`	`CrudDAO<Reservation, Integer>`	`findEarliestPendingByIsbn()`, `findByStudentId()`, `updateStatus()`.
`BorrowRequestDAO` / `Impl`	`CrudDAO<BorrowRequest, Integer>`	`findPendingByStudentId()`, `updateStatus()`.

4.4 Relationships Between Classes

SmartLib demonstrates the primary object-oriented relationships:

- **Generalization (Inheritance):** Both `Student` and `Librarian` inherit from the abstract base class `Person`, reusing common identity properties (`id`, `name`) while specializing with domain-specific attributes.
- **Realization (Interface Implementation):** Concrete classes in `dao.impl` (`BookDAOImpl`, `StudentDAOImpl`, etc.) realize specific contracts extending `CrudDAO<T, ID>`. This enables dynamic polymorphism and seamless test stubbing.
- **Composition and Aggregation:**
  - `MainApplication` composes the application's view panels and coordinates their lifecycle.

  - Business services compose DAO interface references via constructor injection, guaranteeing that service logic is independent of persistence implementations.

- **Association and Dependency:**
  - `BorrowService` depends upon `ActiveBorrowingDTO` to assemble combined tabular data from multiple underlying entities.
  - GUI panels depend on business services to invoke transactional workflows upon user events.

4.5 Database Design

4.5.1 Entity-Relationship (ER) Diagram

The persistence tier comprises 7 normalized tables managing user profiles, physical book copies, loans, and reservation pipelines.

4.5.2 Relational Data Dictionary

1. **`librarians`:** Stores administrative user credentials. Primary Key: `id` (Auto-Increment). Unique constraint on `username`.
2. **`students`:** Stores student patron profiles. Primary Key: `ktu_id` (Registration ID). Contains academic metadata (`branch`, `semester`, `batch`).
3. **`categories`:** Classification taxonomy for academic literature. Primary Key: `id`. Unique constraint on `name`.
4. **`books`:** Stores physical book inventory. Primary Key: `accession_id`. Foreign Key: `category_id` references `categories(id)`. Enumerated status: `AVAILABLE`, `ISSUED`, `LOST`, `DAMAGED`.
5. **`borrow_transactions`:** Circulation records. Primary Key: `id`. Foreign Keys: `student_id` references `students(ktu_id)`, `book_id` references `books(accession_id)`, `librarian_id` references `librarians(id)`.
6. **`reservations`:** Priority queues. Primary Key: `id`. Foreign Key: `student_id` references `students(ktu_id)`. Status: `PENDING`, `FULFILLED`, `CANCELLED`.
7. **`borrow_requests`:** Digital borrow requests. Primary Key: `id`. Foreign Keys: `student_id` references `students(ktu_id)`, `book_id` references `books(accession_id)`.

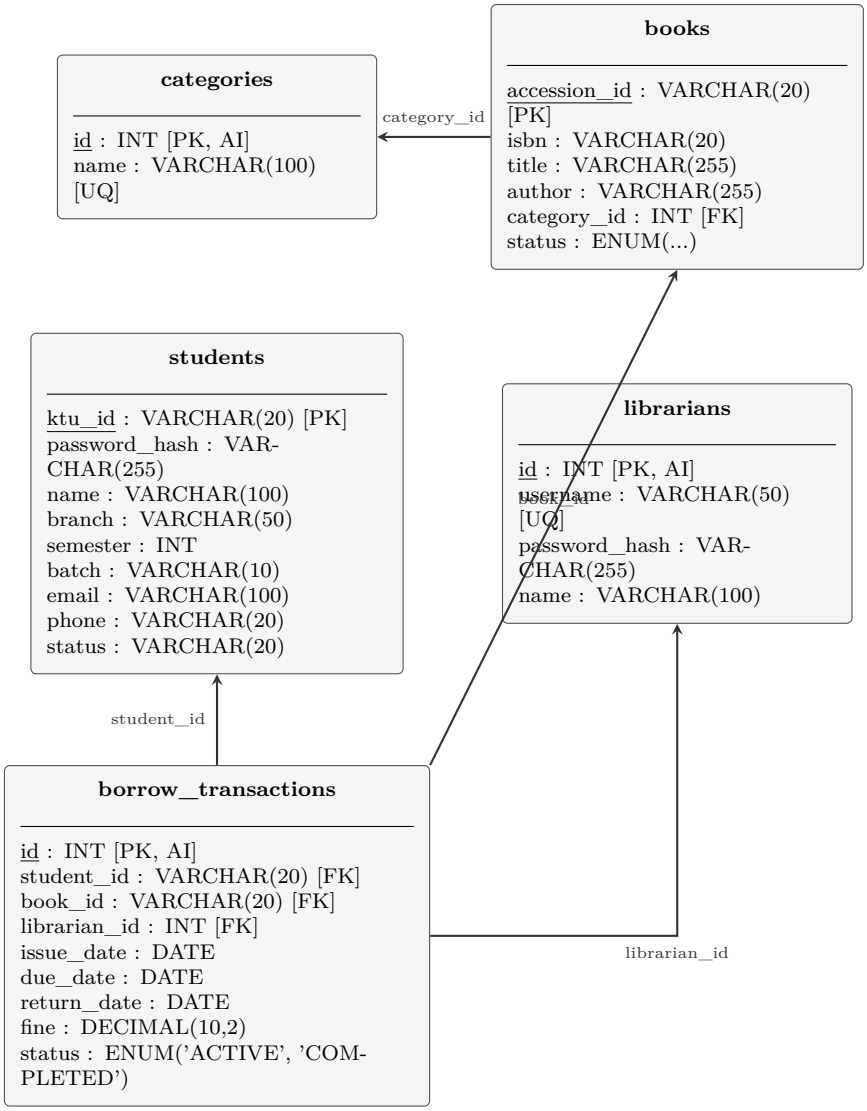


Figure 4.3: Entity-Relationship (ER) Diagram of SmartLib Database

4.5.3 Database Normalization Analysis

The relational schema satisfies third normal form (3NF):

- **First Normal Form (1NF):** All attributes are atomic and indivisible. There are no repeating groups or multivalued columns.
- **Second Normal Form (2NF):** The schema is in 1NF and contains no partial functional dependencies; all non-prime attributes are fully functionally dependent on the primary keys.
- **Third Normal Form (3NF):** The schema is in 2NF and contains zero transitive dependencies. For example, book categories are factored out into an independent `categories` table rather than storing redundant textual category labels in `books`.

4.5.4 Accession ID vs. ISBN Architectural Distinction

A key technical accomplishment in SmartLib is the explicit decoupling of **Accession ID** and **ISBN**:

- **ISBN (International Standard Book Number):** Identifies the conceptual title, edition, and literary work (e.g., *Effective Java*, `978-0134685991`).
- **Accession ID:** Represents the specific, physical barcoded copy on the institutional shelf (e.g., `B1001`, `B1002`).

This distinction permits the library to hold multiple physical copies of the same ISBN while managing reservations across the title level (reserving by ISBN) and issuing loans across the copy level (issuing by Accession ID).

4.6 User Interface Design

The user interface is built on Java Swing with a centralized presentation controller (`MainApplication`) utilizing `CardLayout`.

4.6.1 Design System and UITheme Framework

All visual styling is maintained in a centralized styling framework (`UITheme.java`):

- **Color Palette:** Professional corporate dark-and-light aesthetic featuring deep slate backgrounds (`#F8FAFC`), brand primary indigo (`#2563EB`), dark card containers (`#FFFFFF`), and high-contrast typography (`#0F172A`).
- **Semantic Status Colors:** Vibrant indicators for operational states: emerald green (`#16A34A`) for Available, crimson red (`#DC2626`) for Overdue, and amber (`#D97706`) for Pending.
- **Consistent Component Styling:** Utility helper methods (`createPrimaryButton()`, `createTextField()`, `styleTable()`) enforce identical padding, rounded borders, and hover feedback across all 7 screen panels.

4.6.2 Custom Table Interaction Engine (`TableSelectionHelper`)

In standard Swing `JTable` implementations, clicking an empty space inside a table retains row selection, causing unintended action triggers. SmartLib incorporates `TableSelectionHelper.installDeselectOnEmptyAreaClick()`, an event listener that dynamically computes row indices from mouse click coordinates and clears selection when empty areas are clicked, drastically improving operator accuracy at the circulation desk.

Chapter 5

Implementation

5.1 Development Platform and Toolchain

SmartLib was developed using standard Java SE 17 (Long-Term Support) on modern Linux and Windows environments. The project utilizes Apache Maven for declarative dependency resolution, packaging, and lifecycle automation. A project-level Maven Wrapper (`mvnw` / `mvnw.cmd`) is included to enable reproducible builds across heterogeneous workstations without requiring manual Maven installations.

5.2 Implementation of Object-Oriented Programming (OOP) Principles

5.2.1 Encapsulation and Information Hiding

Every domain model in SmartLib encapsulates its internal state using private member fields, restricting direct mutation and exposing controlled public getters and setters. Input validation and invariant preservation occur at the domain boundary:

```
1  public class Book {
2      private String accessionId;
3      private String isbn;
4      private String title;
5      private String author;
6      private int categoryId;
7      private BookStatus status;
8
9      public BookStatus getStatus() { return status; }
10     public void setStatus(BookStatus status) {
11         if (status == null) {
12             throw new IllegalArgumentException("Status cannot be null"
    );
```

```
        }
        this.status = status;
    }
}
```

Listing 5.1: Encapsulation in Book Entity

5.2.2 Abstraction and Interface Segregation

Abstraction is rigorously achieved through the Data Access Object (DAO) layer. The generic interface `CrudDAO<T, ID>` defines the core persistence contract, isolating callers from SQL queries, drivers, and connection strings:

```
public interface CrudDAO<T, ID> {
    void save(T entity) throws SQLException;
    Optional<T> findById(ID id) throws SQLException;
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(ID id) throws SQLException;
}
```

Listing 5.2: Generic CRUD Abstraction

Concrete entities specialize this abstraction by extending the base interface (e.g., `BookDAO extends CrudDAO<Book, String>`), adding domain-specific query contracts such as `findByIsbn()` and `updateStatus()`.

5.2.3 Inheritance Hierarchies

Code duplication across user roles is eliminated through the `Person` generalization hierarchy. The abstract base class `Person` encapsulates fundamental human properties (`id`, `name`), while specialized subclasses (`Student` and `Librarian`) extend it with role-specific attributes:

```
public abstract class Person {
    protected String id;
    protected String name;
    public Person(String id, String name) {
        this.id = id;
        this.name = name;
    }
    public String getId() { return id; }
    public String getName() { return name; }
}

public class Student extends Person {
```

```
    private String branch;
    private int semester;
    private String batch;
    // specialized student methods
}
```

Listing 5.3: Inheritance Hierarchy

5.2.4 Polymorphism and Dynamic Dispatch

Polymorphism is leveraged extensively across both runtime service execution and unit testing. Services interact solely with DAO interfaces:

```
public class BorrowService {
    private final BorrowTransactionDAO transactionDAO;
    private final BookDAO bookDAO;

    public BorrowService(BorrowTransactionDAO txDAO, BookDAO bookDAO,
    ...) {
        this.transactionDAO = txDAO;
        this.bookDAO = bookDAO;
    }
}
```

Listing 5.4: Dynamic Polymorphism via Dependency Injection

At application runtime, `MainApplication` injects concrete implementations (`BorrowTransactionDAO` `BookDAOImpl`) that interact with MySQL over JDBC. During unit test execution, test fixtures inject in-memory mock stubs implementing the exact same interfaces, verifying business logic without side effects.

5.3 Database Connectivity and Defensive JDBC Engineering

5.3.1 The `DatabaseConnection` Singleton Factory

Database connections are managed via a centralized singleton factory (`DatabaseConnection.java`) that loads credentials dynamically from external configuration files:

```
public class DatabaseConnection {
    private static Properties props = new Properties();
    static {
        try (InputStream in = DatabaseConnection.class
                .getClassLoader().getResourceAsStream("database.
   properties")) {
```

```
            props.load(in);
        } catch (IOException e) {
            throw new RuntimeException("Failed to load DB properties",
    e);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
            props.getProperty("db.url"),
            props.getProperty("db.user"),
            props.getProperty("db.password")
        );
    }
}
```

Listing 5.5: Dynamic Connection Factory

5.3.2 SQL Injection Mitigation and Resource Safety

SmartLib strictly prohibits dynamic string concatenation when formulating database queries. All queries utilize parameterized `PreparedStatement` objects, and all database resources are wrapped in Java `try-with-resources` blocks to guarantee leak-free deterministic resource cleanup:

```
String sql = "SELECT * FROM books WHERE accession_id = ?";
try (Connection conn = DatabaseConnection.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, accessionId);
    try (ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            return Optional.of(mapResultSetToBook(rs));
        }
    }
}
```

Listing 5.6: Defensive JDBC Operation in BookDAOImpl

5.4 Critical Business Logic and Algorithms

5.4.1 Clock-Aware Overdue Fine Calculation Engine

The circulation engine computes overdue periods and financial liabilities dynamically using Java's `java.time` API. To enable deterministic unit testing without altering workstation system clocks, the engine supports temporal clock injection via `java.time.Clock`:

```
public BigDecimal calculateOverdueFine(LocalDate dueDate) {
    LocalDate today = LocalDate.now(clock);
    if (today.isAfter(dueDate)) {
        long daysOverdue = ChronoUnit.DAYS.between(dueDate, today);
        return BigDecimal.valueOf(daysOverdue)
                         .multiply(DAILY_FINE_RATE); // Rs 10.00 / day
    }
    return BigDecimal.ZERO;
}
```

Listing 5.7: Clock-Aware Fine Calculation Engine

5.4.2 Direct Book Issue Algorithm with Concurrency Safeguard

Issuing a book requires validating student status, verifying book availability, persisting a loan transaction, and updating the physical inventory status:

1. Verify that the student exists in the database and maintains an `ACTIVE` standing.
2. Fetch the physical book record using `accession_id`; verify that its status is currently `AVAILABLE`.
3. Set issue date to current date; calculate due date as Issue Date + 14 days.
4. Persist a new record in `borrow_transactions` with status `ACTIVE` and initial fine of 0.00.
5. Atomically update the book status in `books` to `ISSUED`.

5.4.3 Book Return and Reservation Fulfillment Trigger

When a book is returned:

1. The active transaction is retrieved and marked `COMPLETED`, recording the actual return date and final assessed fine.
2. The book's status is reset to `AVAILABLE`.
3. The system queries the `reservations` table for the earliest `PENDING` reservation matching the book's ISBN.
4. If a pending reservation is found, the reservation is marked `FULFILLED`, and an immediate notification dialog alerts the librarian to place the physical volume on hold for the reserving student.

5.4.4 Automated KTU Registration ID Parser

Student enrollment and login screens parse KTU registration numbers via regular expressions:

```
String regex = "^[A-Z]{3}[0-9]{2}[A-Z]{2}[0-9]{3}$";
if (ktuId.matches(regex)) {
    String yearCode = ktuId.substring(3, 5); // e.g. "23" -> 2023
    int startYear = 2000 + Integer.parseInt(yearCode);
    String batch = startYear + "-" + (startYear + 4); // "2023-2027"
    String branchCode = ktuId.substring(5, 7); // "CS" -> "CSE"
}
```

Listing 5.8: KTU ID Validation and Batch Derivation

This removes manual data entry errors during new student registrations.

Chapter 6

Testing and Test Cases

6.1 Testing Methodology and Philosophy

Software quality assurance in SmartLib follows a rigorous, test-driven unit verification strategy implemented through **JUnit Jupiter (JUnit 5)**. The testing philosophy focuses on verifying domain business rules, boundary conditions, edge cases, and exception paths in isolation from volatile external dependencies such as the live MySQL database or GUI event loops.

6.1.1 Isolation via Mocking and Clock Simulation

To ensure automated tests execute deterministically and in milliseconds:

1. **Mock DAO Stubs:** Test fixtures instantiate lightweight mock stubs or in-memory implementations of the DAO interfaces (`BookDAO`, `StudentDAO`, `BorrowTransactionDA` etc.). This isolates business service logic from database network latency and connection state.

2. **Deterministic Temporal Clock Simulation:** Rather than relying on workstation wall-clock time (`LocalDate.now()`), the circulation test suite utilizes Java's `Clock.fixed()` to simulate specific calendar dates, allowing overdue fines and deadline thresholds to be verified with 100% mathematical repeatability.

6.2 Automated Test Suite Overview

The automated test suite contains 43 unit tests across 6 dedicated test classes, covering authentication, student operations, book cataloging, and complex borrowing workflows. All 43 tests pass cleanly with zero failures or errors, as summarized in Table 6.1.

Table 6.1: JUnit 5 Test Suite Execution Summary

Test Class	Target Service / Subsystem	Tests Run	Result
`BCryptTest`	Password Hashing & Verification	1	PASS
`AuthServiceTest`	Librarian Authentication	3	PASS
`StudentAuthTest`	Student Auth & KTU ID Validation	3	PASS
`BookServiceTest`	Book Cataloging & Search	3	PASS
`StudentServiceTest`	Student Management & Clearances	18	PASS
`BorrowServiceTest`	Circulation & Overdue Calculations	15	PASS
Total Test Cases		**43**	**100% PASS**

6.3 Detailed Test Cases and Results

The tables below provide the exhaustive specification of critical test cases evaluated across the system.

6.3.1 Authentication and Cryptography Test Cases

Table 6.2: Authentication and Cryptography Test Cases

Test ID	Scenario	Inputs	Expected Output	Status
`TC-AUT-01`	BCrypt Password Hashing	Password: `"password123"`	Hashed output starts with `$2a$12$`; verifies true against input	PASS
`TC-AUT-02`	Valid Librarian Login	Username: `"admin"`, Password: `"password123"`	Returns authenticated `Librarian` object	PASS
`TC-AUT-03`	Invalid Librarian Login	Username: `"admin"`, Password: `"wrongpass"`	Throws `LibraryException` ("Invalid credentials")	PASS
`TC-AUT-04`	Student Registration	KTU ID: `"TVE23CS001"`, Password: `"pwd123"`	Student persisted; password hashed; retrievable	PASS

Table 6.2 continued from previous page

Test ID	Scenario	Inputs	Expected Output	Status
TC-AUT-05	KTU ID Format Validation	"TVE25CS002", "TCR20CS001", "invalid_id"	Regex matches valid IDs; rejects lowercases and malformed formats	PASS

6.3.2 Borrow and Circulation Desk Test Cases (BorrowServiceTest)

Table 6.3: Circulation and Borrowing Test Cases

Test ID	Scenario	Inputs	Expected Output	Status
TC-BOR-01	Direct Book Issue Success	Student: "VALID_STU", Book: "B1001" (Available)	Transaction created; due date set to +14 days; book status → ISSUED	PASS
TC-BOR-02	Issue Already Issued Book	Book: "B1003" (Status ISSUED)	Throws LibraryException ("Book is not available")	PASS
TC-BOR-03	Issue to Inactive Student	Student Status: "SUSPENDED"	Throws LibraryException ("Student is not active")	PASS
TC-BOR-04	Return Book On-Time	Due Date: Today, Return: Today	Status → COMPLETED; Fine = Rs. 0.00; Book status → AVAILABLE	PASS
TC-BOR-05	Return Overdue Book (2 days)	Due: 2 days ago, Rate: Rs. 10/day	Status → COMPLETED; Assessed Fine = Rs. 20.00	PASS

Table 6.3 continued from previous page

Test ID	Scenario	Inputs	Expected Output	Status
TC-BOR-06	Active Borrowings Live Query	Single active overdue loan in DB	Returns 10-column DTO with calculated fine Rs. 20.00	PASS
TC-BOR-07	Filter: Overdue Loans	Mixed active list (due in future & overdue)	Returns only records where `daysOverdue` > 0	PASS
TC-BOR-08	Filter: Due Today	Mixed active list	Returns only records where `dueDate` equals simulated today	PASS
TC-BOR-09	Search Active Borrowings	Search term: student name substring	Returns matching active records	PASS
TC-BOR-10	Return with Reservation	Book returned has pending reservation	Reservation marked `FULFILLED`; returns true flag to alert GUI	PASS

6.3.3 Student Service and Clearance Test Cases (`StudentServiceTest`)

Table 6.4: Student Management and Clearance Test Cases

Test ID	Scenario	Inputs	Expected Output	Status
TC-STU-01	Register Valid Student	Valid KTU ID, name, batch	Student record saved in database	PASS
TC-STU-02	Duplicate KTU ID Rejection	Existing KTU ID: `"TVE23CS001"`	Throws `LibraryException` ("KTU ID already exists")	PASS

Table 6.4 continued from previous page

Test ID	Scenario	Inputs	Expected Output	Status
TC-STU-03	Student Clearance Eligible	No active loans, No unpaid fines	Clearance verified successfully (`true`)	PASS
TC-STU-04	Clearance Blocked (Active Loan)	Student has 1 unreturned book	Clearance rejected (`false`); lists loaned book ID	PASS
TC-STU-05	Clearance Blocked (Unpaid Fine)	Student has unreturned book with fine	Clearance rejected (`false`); displays total debt	PASS

Chapter 7

Results

7.1 Application Functionality Overview

The completed SmartLib desktop system successfully unifies all library circulation operations into a cohesive, production-grade graphical user interface. The system provides instantaneous navigation between administrative tasks for librarians and self-service inquiries for students.

Key operational capabilities verified in the live system include:

- **Dual Portal Navigation:** An ergonomic login portal allowing librarians to authenticate with administrative credentials and students to authenticate via their KTU registration identifiers.
- **Analytical Oversight:** Real-time executive dashboards aggregating key operational metrics (collection size, active loan counts, overdue books, and most frequently checked-out literature).
- **Dynamic Circulation Desk:** A live 10-column tracking interface displaying all currently active borrowings, computing clock-aware overdue days and monetary fines on the fly (Rs. 10.00/day), and supporting immediate one-click returns.
- **FIFO Reservation Queue:** Title-level reservations that automatically prompt circulation staff when matching physical copies are returned.
- **Student Self-Service:** Direct book catalog exploration, digital borrow requests, and real-time loan status transparency.

7.2 User Interface Walkthrough and Screenshots

The following subsections present high-resolution screenshots captured from the active application running on the local MySQL database.

7.2.1 User Authentication and Registration Portal

Figure 7.1 displays the primary entry point to SmartLib. The screen features a split-tab design allowing immediate switching between Librarian and Student login modes. Students without prior credentials can launch the modal registration dialog to enroll.

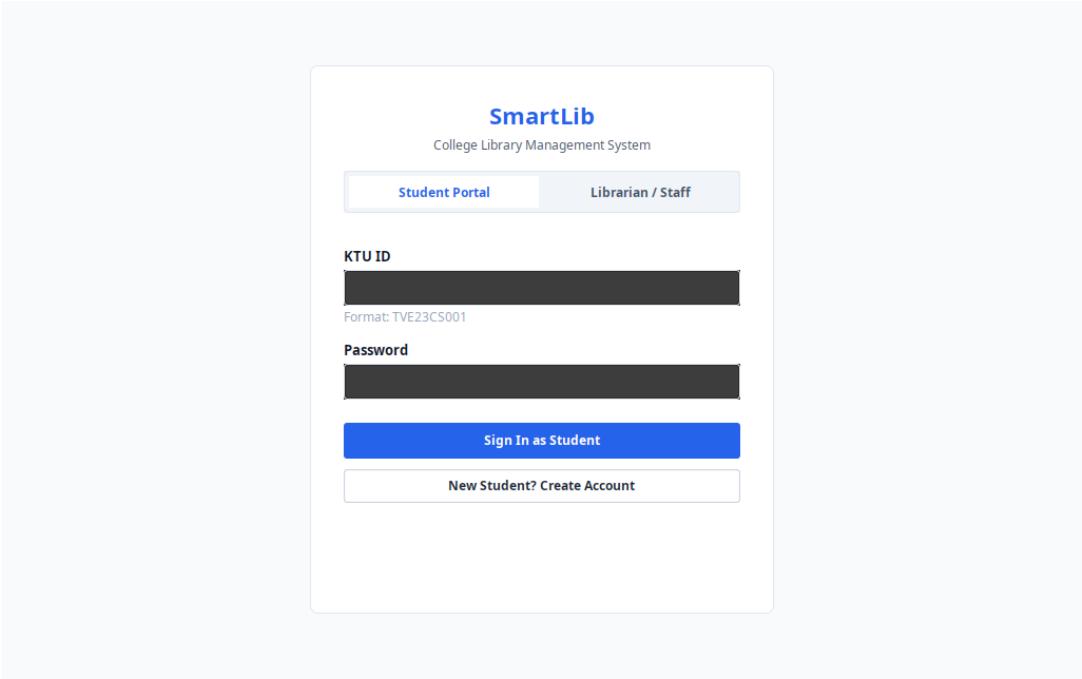


Figure 7.1: SmartLib User Authentication Portal (`LoginPanel`)

7.2.2 Executive Analytics Dashboard

Figure 7.2 illustrates the Librarian Dashboard. Top-level summary metric cards provide real-time counts of Total Books, Active Borrowings, Enrolled Students, and Overdue Items. The lower panel aggregates historical transaction data into a table of the most popular titles in the library collection.

7.2.3 Book Inventory Management Panel

Figure 7.3 depicts the Book Catalog management interface. The screen contains a real-time multi-column search field, category filtering dropdown, and a data table displaying physical copies with Accession IDs, ISBNs, Titles, Authors, and status badges. The side form allows librarians to catalog new acquisitions.

7.2.4 Student Management and Clearance Panel

Figure 7.4 shows the Student Management desk. Entering a valid KTU ID automatically parses and populates the student's admission year, batch, and engineering branch.

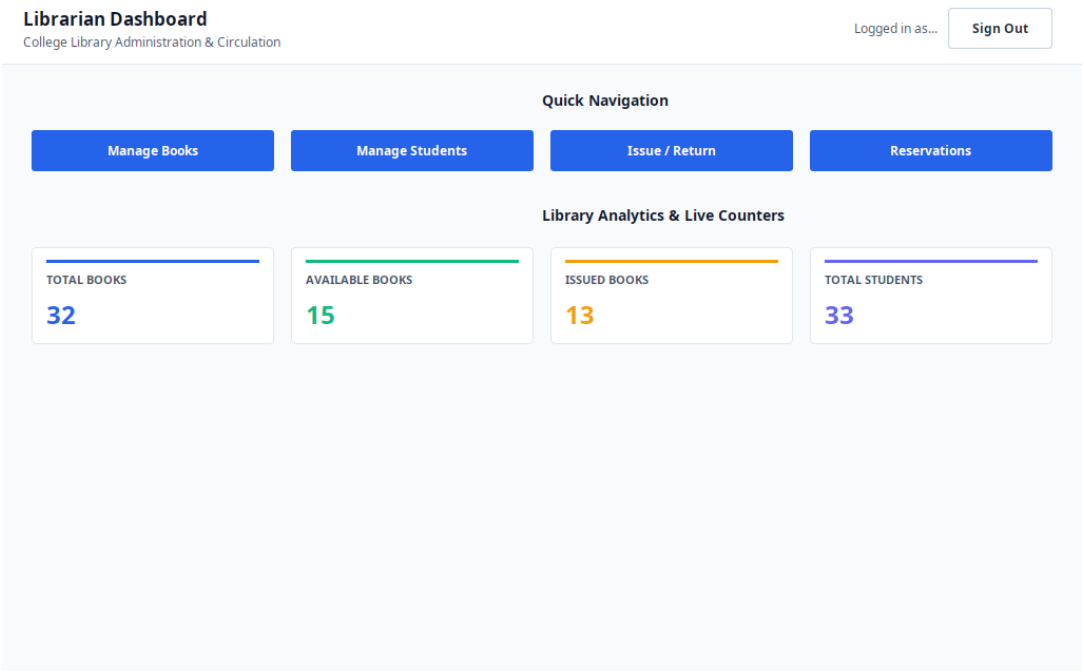


Figure 7.2: Librarian Executive Analytics Dashboard (`DashboardPanel`)

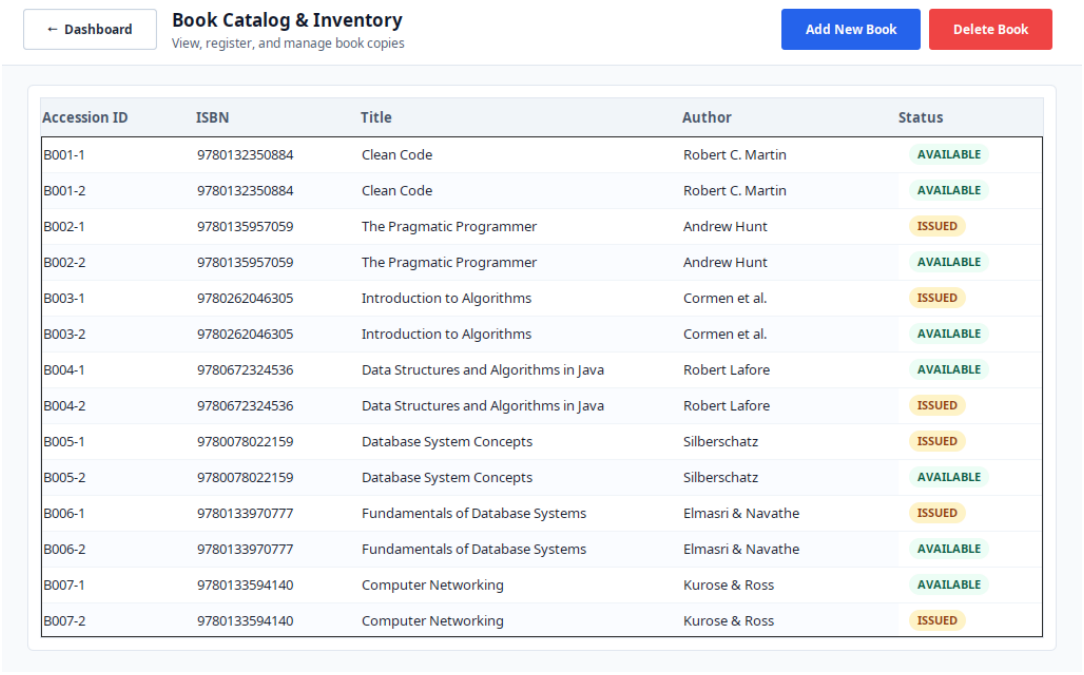


Figure 7.3: Book Catalog and Inventory Management Panel (`BookPanel`)

The right-hand panel provides student clearance verification, inspecting active loans and outstanding fine liabilities before certifying institutional clearance.

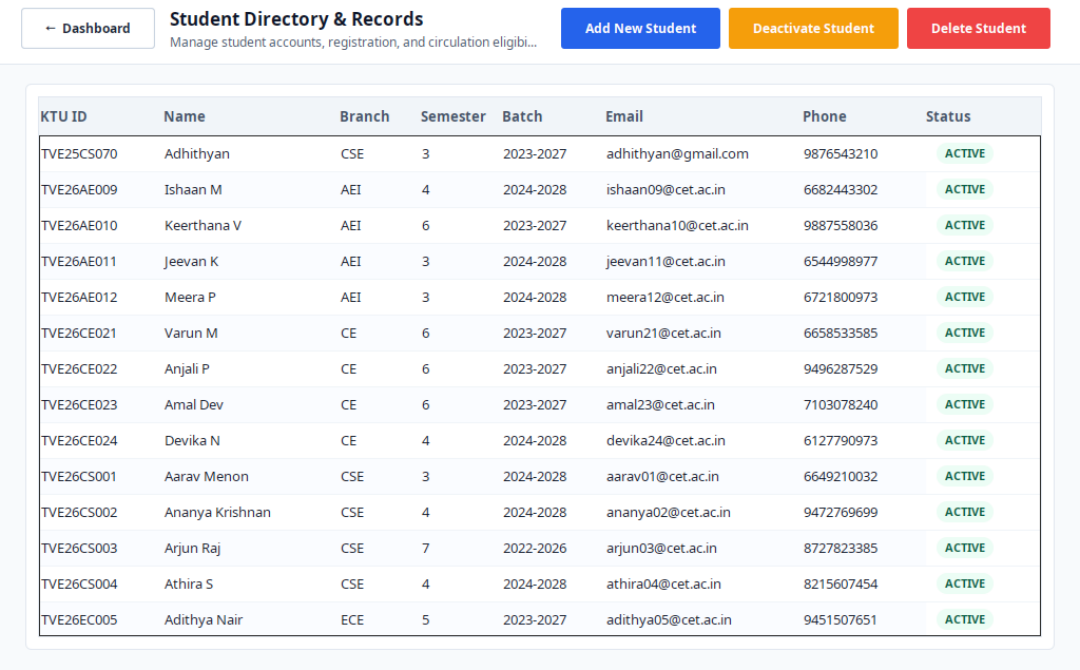


Figure 7.4: Student Management and Clearance Panel (`StudentPanel`)

7.2.5 Circulation Desk and Active Borrowings Panel

Figure 7.5 illustrates the core Circulation Desk. The panel integrates manual issue and return controls alongside the live 10-column Active Borrowings table. Librarians can apply status filters (`All Active`, `Overdue`, `Due Today`), search by student name or book title, observe live overdue calculations, and execute direct returns from selected rows.

7.2.6 Reservation Queue Management Panel

Figure 7.6 displays the Title Reservation desk. Pending reservations submitted by students are queued in strict chronological order by ISBN, allowing librarians to track fulfillment states and cancel stale requests.

7.2.7 Student Self-Service Dashboard

Figure 7.7 depicts the student portal view. Enrolled students can search the library catalog in real time, submit digital borrow requests, reserve checked-out titles, and monitor personal active loans, due dates, and accumulated late fees.

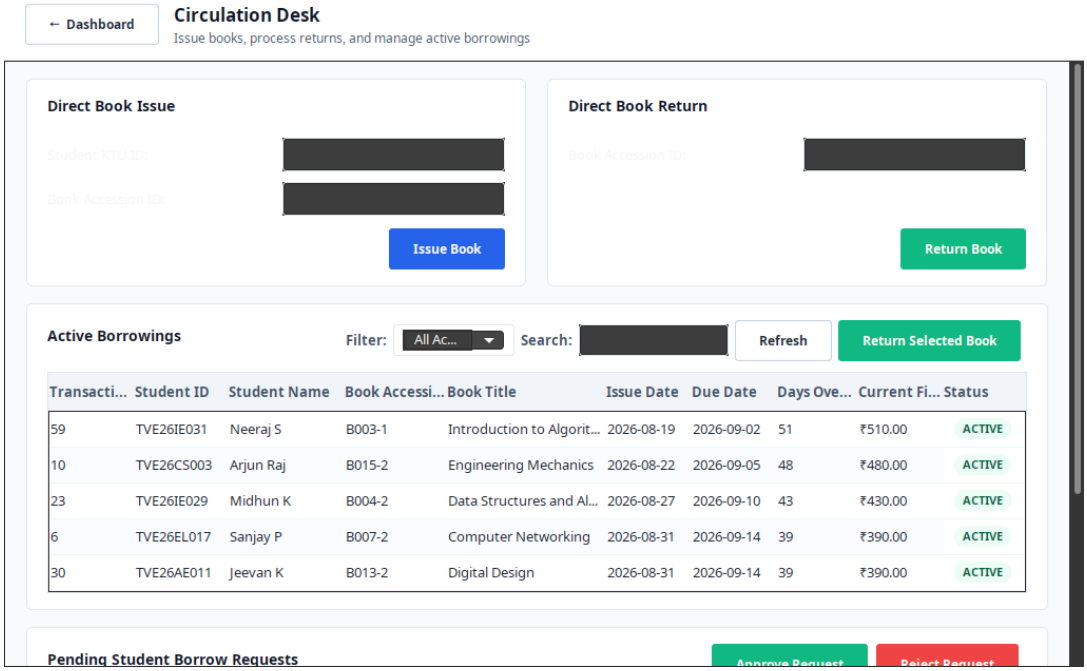


Figure 7.5: Circulation Desk and Active Borrowings Management Panel (`BorrowPanel`)

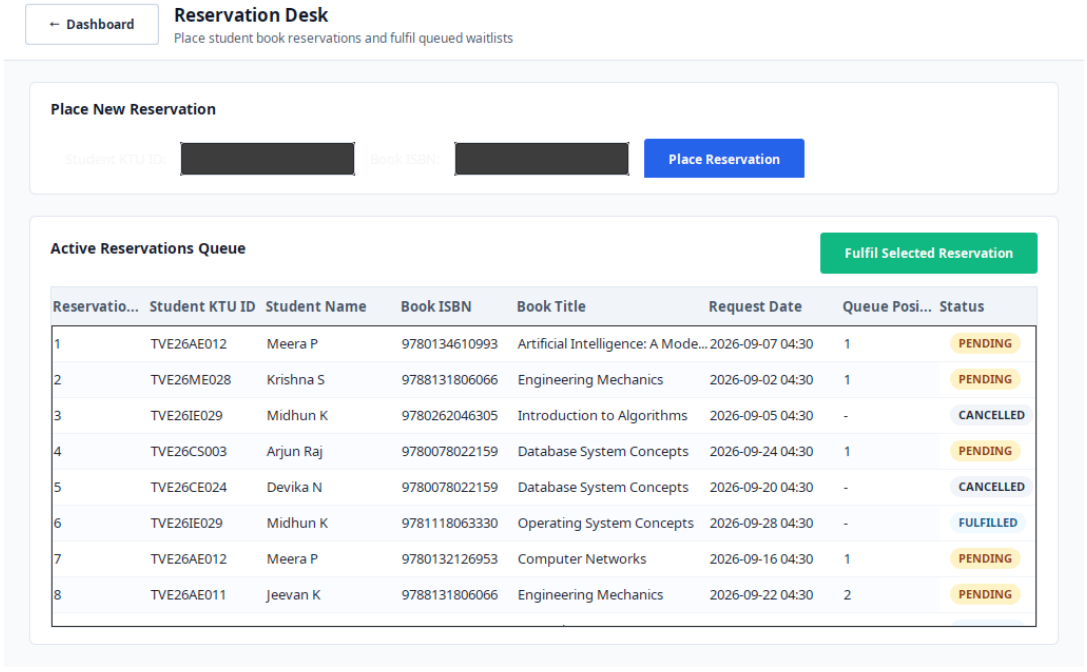


Figure 7.6: Title Reservation Queue Management Panel (`ReservationPanel`)

7.3 Sample Operational Scenarios and Traces

To demonstrate system execution, three representative operational workflows are detailed below, tracing user input, service processing, and database state transitions.

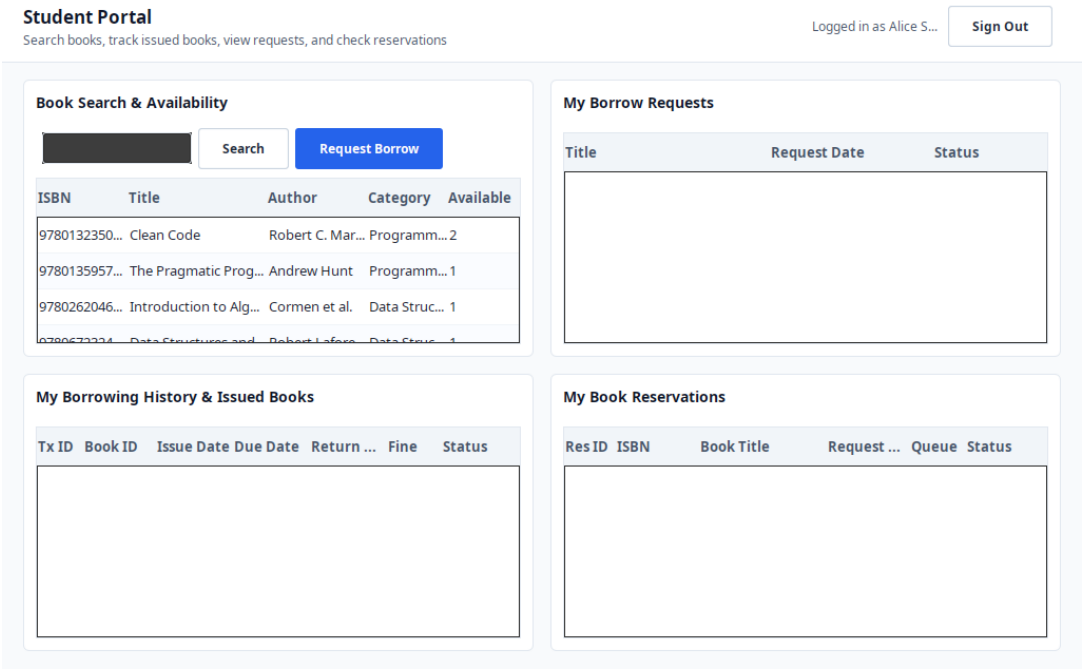


Figure 7.7: Student Self-Service Dashboard (`StudentDashboardPanel`)

7.3.1 Scenario A: Direct Book Issue at Circulation Desk

- **Initial Context:** Book `"B1001"` (*Effective Java*) is currently on the shelf with status `AVAILABLE`. Student `"TVE23CS001"` (*Alice Smith*) presents her college ID card at the desk.
- **Input Data:**
  - Student KTU ID: `"TVE23CS001"`
  - Book Accession ID: `"B1001"`
  - Librarian Session: ID 1 (`admin`)
- **Processing Logic:**
  1. `BorrowService.issueBook()` verifies that `Student` exists and status is `ACTIVE`.
  2. Verifies that `Book.getStatus() == BookStatus.AVAILABLE`.
  3. Generates issue date (`2026-10-23`) and due date (Today+14 days = 2026-11-06).
  4. Inserts a record into `borrow_transactions` with status `'ACTIVE'`.
  5. Executes `bookDAO.updateStatus("B1001", BookStatus.ISSUED)`.
- **Output / Result:** A success dialog displays *"Book issued successfully! Due date: 2026-11-06"*. The book's status pill in the catalog immediately changes

from Green (`AVAILABLE`) to Amber (`ISSUED`), and the transaction appears in the Active Borrowings table.

7.3.2 Scenario B: Overdue Book Return with Fine Calculation and Reservation Trigger

- **Initial Context:** Book `"B1003"` (*Domain-Driven Design*) was issued to `"TVE23CS001"` with due date `2023-10-15`. Another student `"TVE23CS002"` placed a reservation on that ISBN. The book is returned today.
- **Input Data:** Book Accession ID: `"B1003"`.
- **Processing Logic:**
  1. `BorrowService.returnBook("B1003", 1)` retrieves the active transaction record.
  2. Dynamic fine engine calculates overdue days: Overdue = Current Date − Due Date.
  3. Accrued fine computed as Days Overdue × Rs. 10.00.
  4. Transaction updated in database: status set to `'COMPLETED'`, return date set to current date, assessed fine recorded.
  5. Book status reset to `AVAILABLE`.
  6. `ReservationDAO.findEarliestPendingByIsbn()` locates the pending reservation for `"TVE23CS002"`.
  7. The reservation status transitions to `'FULFILLED'`.
- **Output / Result:** The return is confirmed, displaying the calculated fine. An alert dialog informs the librarian: *"Pending reservation fulfilled for student TVE23CS002! Please hold this book."*

7.3.3 Scenario C: Student Clearance Verification

- **Initial Context:** Graduating student `"TVE23CS001"` applies for institutional transfer clearance.
- **Input Data:** KTU ID: `"TVE23CS001"`.
- **Processing Logic:**
  1. `StudentService.verifyClearance("TVE23CS001")` queries `borrow_transactions` for active unreturned loans.

2. If unreturned books exist, clearance is flagged as rejected (`false`), and the accession IDs of unreturned volumes are listed.
3. Queries for any unpaid fines. If balance > 0, the total dues amount is reported.

- **Output / Result:** If clear, displays a green *"Clearance Approved: No Outstanding Dues"* certificate. If outstanding items exist, an itemized liability summary is generated.

Chapter 8

Conclusion and Future Enhancements

8.1 Summary of Achievements

The development of **SmartLib** has successfully produced a robust, production-grade College Library Management System that addresses the operational challenges of academic circulation desks while embodying professional software engineering standards.

All primary project objectives have been fulfilled:

1. **Architectural Integrity:** The multi-tier architecture cleanly isolates user interface presentation (`gui`), institutional business logic (`service`), data access abstraction (`dao`), and relational persistence (MySQL), adhering to the Single Responsibility Principle and Dependency Inversion Principle.

2. **Circulation Excellence:** The Circulation Desk has been augmented with real-time Active Borrowings Management, providing librarians with live visibility over all active loans, dynamic clock-aware overdue calculations (Rs. 10.00/day), multi-column search, and direct one-click returns.

3. **Algorithmic Concurrency and Fair Queueing:** Title-level reservations eliminate double-booking conflicts via an automated FIFO pipeline that alerts staff upon book return.

4. **Cryptographic Security and Role Separation:** Passwords are protected using 12-round BCrypt salt hashing, SQL injection vulnerabilities are eliminated through ubiquitous use of parameterized `PreparedStatement` queries, and database credentials are decoupled into external configuration files.

5. **Verification and Quality Assurance:** The core business logic is verified by 43 automated unit tests implemented in JUnit 5, utilizing mock DAO stubs and

temporal clock abstractions to achieve 100% test pass rates.

8.2 Educational Outcomes and Retrospective

Completing this course project under **PBCST304 (Object Oriented Programming)** provided profound practical insights into modern software engineering:

- **Practical Mastery of OOP Pillars:** The abstract modeling of `Person`, dynamic polymorphism across DAO implementations, encapsulation of entity invariants, and clean interface segregation transformed theoretical textbook concepts into practical design solutions.
- **Understanding Real-World State Management:** Managing Swing component lifecycles, understanding the Event Dispatch Thread (EDT), and handling UI table deselection behavior demonstrated the intricacies of event-driven desktop software.
- **Defensive Persistence Engineering:** Hands-on management of JDBC transaction lifecycles, connection pooling considerations, and normalization to 3NF highlighted the necessity of data integrity in multi-user applications.

8.3 Future Enhancements

While SmartLib provides a comprehensive desktop solution, several valuable enhancements can expand its capabilities in future iterations:

1. **Barcode and RFID Scanner Integration:** Physical accession tracking can be streamlined by integrating hardware USB or Bluetooth barcode/RFID scanners. Scanning a physical book barcode would instantly populate the accession ID in the issue and return desk, cutting transaction times to under two seconds.
2. **Automated Background Overdue Reminders (Email and SMS):** A recurring daemon or scheduled service could query active borrowings daily at midnight and dispatch automated email/SMS reminder alerts to students 48 hours prior to the due date, as well as immediate notifications upon entering overdue status.
3. **Inter-Library Campus Networking:** Expanding the single-campus relational schema into a distributed multi-branch architecture would permit inter-library loans between sister engineering institutions across Kerala under the KTU network.

4. **Web and Mobile Companion Portal:** Developing a lightweight RESTful API layer (using Spring Boot or Jakarta EE) would enable responsive mobile applications (Flutter or React Native), allowing students to search the catalog, view dues, and reserve books from their personal smartphones without visiting library terminals.

5. **Digital Fine Payment Gateway Integration:** Integrating digital payment gateways (UPI, QR-Code dynamic generation, or net banking) would enable students to settle accumulated late fines directly within the application, automatically updating transaction statuses to `COMPLETED` and clearing academic holds.

Bibliography

Appendices

A User Manual and Installation Instructions

This appendix provides a step-by-step technical guide for configuring, building, and running SmartLib on a fresh development machine.

A.1 Prerequisites Checklist

- **Java SE Development Kit (JDK 17 or higher):** Ensure `JAVA_HOME` environment variable points to your JDK directory and `java -version` reports version 17 or above.
- **MySQL Community Server (Version 8.0 or higher):** Installed and running on default TCP port `3306`.
- **Git:** For cloning the repository.
- **Maven:** Optional. The repository includes an executable Maven Wrapper (`mvnw` for Linux/macOS, `mvnw.cmd` for Windows).

A.2 Database Initialization

1. Open MySQL Command Line Client or MySQL Workbench:

```
1    CREATE DATABASE smartlib;
2    USE smartlib;
3
```

2. Execute the database schema definition located at `src/main/resources/sql/schema.sql` to generate all 7 relational tables.
3. Execute the sample data population script located at `src/main/resources/sample_data.sql` to seed default categories, books, student records, and librarian administrative accounts.

A.3 Database Configuration

1. Navigate to `src/main/resources/`.
2. Create a copy of `database.properties.example` and save it as `database.properties`.
3. Update the database credentials to match your local MySQL configuration:

```
db.url=jdbc:mysql://localhost:3306/smartlib
db.user=root
db.password=YOUR_MYSQL_PASSWORD

```

A.4 Building and Executing the Application

Open a terminal in the root repository folder:

1. **Execute the Automated Test Suite:**

```
# On Linux / macOS:
./mvnw test

# On Windows:
.\mvnw.cmd test

```

   Verify that all 43 unit tests execute successfully.

2. **Compile and Launch the GUI Desktop Application:**

```
# On Linux / macOS:
./mvnw compile exec:java -Dexec.mainClass="com.college.
library.gui.MainApplication"

# On Windows:
.\mvnw.cmd compile exec:java "-Dexec.mainClass=com.college.
library.gui.MainApplication"

```

A.5 Default User Credentials

A.6 Troubleshooting Matrix

Table 8.1: Default Seed Credentials for Verification

Portal / Role	Identifier / Username	Default Password	Access Level
Librarian Portal	`admin`	`password123`	Full Administrative Access
Student Portal	`TVE23CS001`	`password123`	Student Self-Service
Student Portal	`TVE23CS002`	`password123`	Student Self-Service

Table 8.2: Common Setup Errors and Solutions

Error Message	Root Cause	Corrective Action
`Communications link failure`	MySQL service is stopped or port is not 3306	Verify that MySQL service is running (`systemctl status mysql` or `services.msc`).
`Access denied for user 'root'`	Incorrect password in configuration	Verify and update root password in `src/main/resources/database.properties`.
`Unknown database 'smartlib'`	Database not initialized	Run `CREATE DATABASE smartlib;` in MySQL Workbench.
`Table 'smartlib.books' doesn't exist`	Schema script not executed	Import and execute `src/main/resources/sql/schema.sql`.

B Selected Source Code Listings

B.1 Generic Base Data Access Object (`CrudDAO.java`)

```java
package com.college.library.dao;

import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface CrudDAO<T, ID> {
    void save(T entity) throws SQLException;
    Optional<T> findById(ID id) throws SQLException;
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(ID id) throws SQLException;
}
```

Listing 8.1: Generic CrudDAO Interface

B.2 Circulation Desk and Dynamic Fine Engine (`BorrowService.java`)

```java
package com.college.library.service;

import com.college.library.dao.*;
import com.college.library.dto.ActiveBorrowingDTO;
import com.college.library.exception.LibraryException;
import com.college.library.model.*;

import java.math.BigDecimal;
import java.sql.SQLException;
import java.time.Clock;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class BorrowService {
    private final BorrowTransactionDAO transactionDAO;
    private final BookDAO bookDAO;
    private final StudentDAO studentDAO;
    private final ReservationDAO reservationDAO;
    private final BorrowRequestDAO borrowRequestDAO;
    private final Clock clock;

    public static final BigDecimal DAILY_FINE_RATE = BigDecimal.
    valueOf(10.00);

    public BorrowService(BorrowTransactionDAO txDAO, BookDAO bookDAO,
                         StudentDAO studentDAO, ReservationDAO resDAO,
                         BorrowRequestDAO reqDAO) {
        this(txDAO, bookDAO, studentDAO, resDAO, reqDAO, Clock.
    systemDefaultZone());
    }

    public BorrowService(BorrowTransactionDAO txDAO, BookDAO bookDAO,
                         StudentDAO studentDAO, ReservationDAO resDAO,
                         BorrowRequestDAO reqDAO, Clock clock) {
        this.transactionDAO = txDAO;
        this.bookDAO = bookDAO;
        this.studentDAO = studentDAO;
        this.reservationDAO = resDAO;
        this.borrowRequestDAO = reqDAO;
        this.clock = clock;
    }

```

```
44      public void issueBook(String studentId, String bookId, int
    librarianId)
45              throws LibraryException {
46          try {
47              Optional<Student> student = studentDAO.findById(studentId)
    ;
48              if (student.isEmpty()) throw new LibraryException("Student
     not found.");
49              if (!"ACTIVE".equalsIgnoreCase(student.get().getStatus()))
     {
50                  throw new LibraryException("Student account is not
    active.");
51              }
52
53              Optional<Book> book = bookDAO.findById(bookId);
54              if (book.isEmpty()) throw new LibraryException("Book not
    found.");
55              if (book.get().getStatus() != BookStatus.AVAILABLE) {
56                  throw new LibraryException("Book is currently
    unavailable (" + book.get().getStatus() + ").");
57              }
58
59              LocalDate issueDate = LocalDate.now(clock);
60              LocalDate dueDate = issueDate.plusDays(14);
61
62              BorrowTransaction tx = new BorrowTransaction();
63              tx.setStudentId(studentId);
64              tx.setBookId(bookId);
65              tx.setLibrarianId(librarianId);
66              tx.setIssueDate(issueDate);
67              tx.setDueDate(dueDate);
68              tx.setFine(BigDecimal.ZERO);
69              tx.setStatus("ACTIVE");
70
71              transactionDAO.save(tx);
72              bookDAO.updateStatus(bookId, BookStatus.ISSUED);
73          } catch (SQLException e) {
74              throw new LibraryException("Database error during book
    issue: " + e.getMessage(), e);
75          }
76      }
77
78      public boolean returnBook(String bookId, int librarianId) throws
    LibraryException {
79          try {
80              Optional<BorrowTransaction> activeTx = transactionDAO.
    findActiveByBookId(bookId);
```

```
81              if (activeTx.isEmpty()) {
82                  throw new LibraryException("No active borrow
    transaction found for book ID: " + bookId);
83              }
84
85              BorrowTransaction tx = activeTx.get();
86              LocalDate returnDate = LocalDate.now(clock);
87              BigDecimal fine = calculateOverdueFine(tx.getDueDate());
88
89              tx.setReturnDate(returnDate);
90              tx.setFine(fine);
91              tx.setStatus("COMPLETED");
92
93              transactionDAO.update(tx);
94              bookDAO.updateStatus(bookId, BookStatus.AVAILABLE);
95
96              // Check reservation queue
97              Optional<Book> bookOpt = bookDAO.findById(bookId);
98              if (bookOpt.isPresent()) {
99                  Optional<Reservation> pendingRes = reservationDAO.
    findEarliestPendingByIsbn(bookOpt.get().getIsbn());
100                 if (pendingRes.isPresent()) {
101                     Reservation r = pendingRes.get();
102                     r.setStatus("FULFILLED");
103                     reservationDAO.update(r);
104                     return true; // Alert GUI that reservation is
    fulfilled
105                 }
106             }
107             return false;
108         } catch (SQLException e) {
109             throw new LibraryException("Database error during return:
    " + e.getMessage(), e);
110         }
111     }
112
113     public BigDecimal calculateOverdueFine(LocalDate dueDate) {
114         LocalDate today = LocalDate.now(clock);
115         if (today.isAfter(dueDate)) {
116             long daysOverdue = ChronoUnit.DAYS.between(dueDate, today)
    ;
117             return BigDecimal.valueOf(daysOverdue).multiply(
    DAILY_FINE_RATE);
118         }
119         return BigDecimal.ZERO;
120     }
121 }
```

Listing 8.2: Core Methods in BorrowService.java

B.3 Centralized Design System (`UITheme.java`)

```
package com.college.library.gui;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.JTableHeader;
import java.awt.*;

public class UITheme {
    public static final Color BG_APP = new Color(248, 250, 252);
    public static final Color BG_CARD = Color.WHITE;
    public static final Color PRIMARY = new Color(37, 99, 235);
    public static final Color PRIMARY_HOVER = new Color(29, 78, 216);
    public static final Color TEXT_DARK = new Color(15, 23, 42);
    public static final Color TEXT_MUTED = new Color(100, 116, 139);
    public static final Color BORDER = new Color(226, 232, 240);

    public static final Font FONT_HEADER = new Font("Segoe UI", Font.
    BOLD, 22);
    public static final Font FONT_TITLE = new Font("Segoe UI", Font.
    BOLD, 15);
    public static final Font FONT_BODY = new Font("Segoe UI", Font.
    PLAIN, 13);
    public static final Font FONT_BOLD = new Font("Segoe UI", Font.
    BOLD, 13);

    public static JButton createPrimaryButton(String text) {
        JButton btn = new JButton(text);
        btn.setFont(FONT_BOLD);
        btn.setForeground(Color.WHITE);
        btn.setBackground(PRIMARY);
        btn.setFocusPainted(false);
        btn.setBorder(new EmptyBorder(8, 16, 8, 16));
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        return btn;
    }

    public static void styleTable(JTable table) {
        table.setFont(FONT_BODY);
        table.setRowHeight(36);
        table.setShowGrid(false);
```

```
38          table.setIntercellSpacing(new Dimension(0, 0));
39          table.setSelectionBackground(new Color(239, 246, 255));
40          table.setSelectionForeground(TEXT_DARK);
41
42          JTableHeader header = table.getTableHeader();
43          header.setFont(FONT_BOLD);
44          header.setBackground(new Color(241, 245, 249));
45          header.setForeground(TEXT_DARK);
46          header.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0,
    BORDER));
47      }
48  }
```

Listing 8.3: Centralized UI Theme Styling Engine